

Ejercicios Prácticos - Unidad 3, Sesión 1

Arquitectura Transformer en Profundidad

Ejercicio 1: Cálculo Manual de Self-Attention

Metadata

- **Duración estimada:** 30 minutos
- **Tipo:** Cálculo/Análisis
- **Modalidad:** Individual
- **Dificultad:** Intermedia
- **Prerrequisitos:** Lectura de teoría sobre el mecanismo de atención, álgebra de matrices básica

Contexto

El mecanismo de self-attention es el corazón de la arquitectura Transformer. Para entender realmente cómo funciona, no hay mejor camino que calcular cada paso a mano con matrices pequeñas. Este ejercicio te permitirá desmitificar la "magia" detrás de la atención y ver exactamente cómo los tokens se relacionan entre sí.

Objetivo de Aprendizaje

- Comprender el flujo completo del cálculo de self-attention
- Calcular manualmente las matrices de scores, pesos y salida
- Interpretar qué representan los pesos de atención en términos de relaciones entre tokens
- Desarrollar intuición sobre cómo la escala por raíz de d_k afecta la distribución de softmax

Enunciado

Dadas las siguientes matrices de Query (Q), Key (K) y Value (V) para una secuencia de 3 tokens con dimensión $d_k = 2$:

```
Q = [[1, 0],  
     [0, 1],  
     [1, 1]]
```

```
K = [[1, 0],
      [0, 1],
      [0.5, 0.5]]
```

```
V = [[1, 2],
      [3, 4],
      [5, 6]]
```

Realiza los siguientes pasos mostrando **todas las matrices intermedias** con al menos 4 decimales de precisión:

Paso 1: Calcular los Scores (QK^T)

Multiplica la matriz Q por la transpuesta de K para obtener la matriz de scores sin escalar.

```
Scores = Q * K^T
```

Recuerda que K^T (la transpuesta de K) es:

```
K^T = [[1, 0, 0.5],
       [0, 1, 0.5]]
```

Completa la matriz resultante (3x3):

```
Scores = [[_, _, _],
          [_, _, _],
          [_, _, _]]
```

Pregunta: Cada elemento `scores[i][j]` representa la compatibilidad entre el token i (como query) y el token j (como key). Cuál par de tokens tiene mayor compatibilidad según los scores sin escalar?

Paso 2: Escalar por raíz de d_k

Divide cada elemento de la matriz de scores por $\sqrt{d_k}$, donde $d_k = 2$.

```
sqrt(2) = 1.4142
```

```
Scaled_Scores = Scores / sqrt(2)
```

Completa la matriz resultante:

```
Scaled_Scores = [[_, _, _],
                  [_, _, _],
                  [_, _, _]]
```

Pregunta: Por qué escalamos por $\sqrt{d_k}$? Qué pasaría con los gradientes si no lo hiciéramos?

Paso 3: Aplicar Softmax por filas

Para cada fila de la matriz escalada, aplica la función softmax:

```
softmax(x_i) = exp(x_i) / sum(exp(x_j)) para todo j en la fila
```

Calcula la softmax para cada fila individualmente:

Fila 1 (correspondiente al Token 1 como query):

```
exp(scaled_scores[0][0]) = exp(__) = __
exp(scaled_scores[0][1]) = exp(__) = __
exp(scaled_scores[0][2]) = exp(__) = __
Suma = __
Softmax = [__, __, __]
```

Fila 2 (correspondiente al Token 2 como query):

```
exp(scaled_scores[1][0]) = exp(__) = __
exp(scaled_scores[1][1]) = exp(__) = __
exp(scaled_scores[1][2]) = exp(__) = __
Suma = __
Softmax = [__, __, __]
```

Fila 3 (correspondiente al Token 3 como query):

```
exp(scaled_scores[2][0]) = exp(__) = __
exp(scaled_scores[2][1]) = exp(__) = __
exp(scaled_scores[2][2]) = exp(__) = __
Suma = __
Softmax = [__, __, __]
```

Matriz de pesos de atención completa:

```
Attention_Weights = [[__, __, __],
                     [__, __, __],
                     [__, __, __]]
```

Pregunta: Verifica que cada fila suma 1.0. Por qué es importante esta propiedad?

Paso 4: Multiplicar por V

Finalmente, multiplica la matriz de pesos de atención por V para obtener la salida:

```
Output = Attention_Weights * V
```

Completa la matriz resultante (3x2):

```
Output = [ [__, __],  
           [__, __],  
           [__, __] ]
```

Preguntas de Reflexión

1. Compara la salida del Token 3 con los valores originales V. El Token 3 (query = [1, 1]) tiene compatibilidad alta con todos los keys. Cómo se refleja esto en su vector de salida?
 2. Si cambiamos Q[0] de [1, 0] a [10, 0], cómo cambiarían los pesos de atención de la primera fila? Qué relación tiene esto con el escalado por $\sqrt{d_k}$?
 3. En un Transformer real, Q, K y V se obtienen mediante proyecciones lineales de la entrada (Q = XW_Q , etc.). Por qué es ventajoso tener proyecciones separadas en lugar de usar directamente las embeddings?
-

Ejercicio 2: Análisis de Arquitecturas Transformer

Metadata

- **Duración estimada:** 20 minutos
- **Tipo:** Clasificación/Análisis
- **Modalidad:** Individual
- **Dificultad:** Básica
- **Prerrequisitos:** Lectura de teoría sobre variantes de la arquitectura Transformer (encoder-only, decoder-only, encoder-decoder)

Contexto

Desde la publicación de "Attention Is All You Need" en 2017, la arquitectura Transformer ha dado lugar a múltiples variantes. Cada variante tiene fortalezas particulares según el tipo de tarea. Comprender cómo se clasifican los modelos más importantes y por qué fueron diseñados de cierta manera es fundamental para cualquier profesional de ML.

Objetivo de Aprendizaje

- Clasificar modelos reales según su tipo de arquitectura Transformer
- Relacionar el tipo de arquitectura con el caso de uso principal
- Analizar tendencias en la evolución de las arquitecturas

- Comprender por qué el paradigma decoder-only ha dominado en los últimos años

Enunciado

Parte A: Clasificación de Modelos (10 min)

Completa la siguiente tabla clasificando cada modelo en su tipo de arquitectura y su caso de uso principal:

Modelo	Organización	Tipo de Arquitectura	Caso de Uso Principal
BERT	Google	—	
GPT-2	OpenAI	—	
GPT-4	OpenAI	—	
T5	Google	—	
Claude 3.5	Anthropic	—	
LLaMA 3	Meta	—	
BART	Meta (Facebook AI)	—	
RoBERTa	Meta (Facebook AI)	—	
Mistral 7B	Mistral AI	—	
Gemini	Google DeepMind	—	
ALBERT	Google	—	

Whisper	OpenAI		

Tipos de arquitectura posibles:

- Encoder-only
- Decoder-only
- Encoder-decoder

Casos de uso principales sugeridos:

- Comprensión/clasificación de texto
- Generación de texto
- Generación de texto conversacional y de propósito general
- Traducción / tareas seq2seq
- Resumen y generación condicional
- Comprensión eficiente de texto
- Reconocimiento automático de habla (ASR)

Parte B: Análisis de Tendencias (5 min)

Responde las siguientes preguntas:

1. **Cuenta:** De los 12 modelos listados, cuántos son de cada tipo?
 - Encoder-only: ____
 - Decoder-only: ____
 - Encoder-decoder: ____
2. **Tendencia temporal:** Los modelos más antiguos (BERT, GPT-2, 2018-2019) incluyen encoder-only y decoder-only. Los modelos más recientes (GPT-4, Claude, LLaMA, Mistral, Gemini, 2023-2024) son casi todos decoder-only. A qué crees que se debe esta convergencia?
3. **Escala vs. arquitectura:** BERT-base tiene ~110M parámetros, mientras que GPT-4 tiene estimados ~1.8T parámetros. La diferencia en rendimiento se debe solo a la escala o la arquitectura también juega un papel? Argumenta tu respuesta.

Parte C: Preguntas de Profundidad (5 min)

1. **Enmascaramiento causal:** Los modelos decoder-only usan atención causal (masked self-attention), donde cada token solo puede atender a tokens previos. Por qué esta restricción es necesaria para generación de texto? Y por qué BERT no la necesita?
2. **Encoder-decoder vs. Decoder-only para traducción:** T5 (encoder-decoder) fue diseñado explícitamente para tareas seq2seq como traducción. Sin embargo, GPT-4 (decoder-only) también puede traducir con alta calidad. Cómo logra un decoder-only realizar tareas que originalmente se diseñaron para encoder-decoder? Qué compromiso existe?

3. **Whisper como caso especial:** Whisper usa una arquitectura encoder-decoder pero para audio-a-texto. El encoder procesa espectrogramas de audio y el decoder genera texto. Por qué tiene sentido una arquitectura encoder-decoder para esta tarea en particular, en lugar de decoder-only?
-

Ejercicio 3: Visualización de Atención con BertViz

Metadata

- **Duración estimada:** 30 minutos
- **Tipo:** Programación/Exploración
- **Modalidad:** Individual
- **Dificultad:** Intermedia
- **Prerrequisitos:** Python básico, familiaridad con pip install, comprensión del mecanismo de atención

Contexto

Ver los patrones de atención en un modelo real es una de las formas más intuitivas de entender qué aprenden las diferentes cabezas de atención. BertViz es una herramienta de visualización que nos permite inspeccionar exactamente a qué tokens atiende cada cabeza en cada capa del modelo.

Objetivo de Aprendizaje

- Configurar y usar BertViz para visualizar patrones de atención
- Identificar qué relaciones lingüísticas capturan diferentes cabezas de atención
- Analizar diferencias en patrones entre capas tempranas y profundas
- Desarrollar intuición sobre cómo el modelo procesa el lenguaje

Enunciado

Parte A: Instalación y Configuración (5 min)

Instala las librerías necesarias:

```
pip install bertviz transformers torch
```

Ejecuta el siguiente código para verificar que todo funciona:

```
from bertviz import head_view, model_view
from transformers import AutoTokenizer, AutoModel
```

```

import torch

# Cargar modelo y tokenizer
model_name = "bert-base-multilingual-cased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name,
output_attentions=True)

print("Modelo cargado correctamente")
print(f"Capas: {model.config.num_hidden_layers}")
print(f"Cabezas de atención por capa:
{model.config.num_attention_heads}")
print(f"Dimensión del modelo: {model.config.hidden_size}")

```

Parte B: Visualización Básica (10 min)

Usa el siguiente código para visualizar la atención en una oración en español:

```

def visualize_attention(sentence, model, tokenizer):
    """Visualiza los patrones de atención para una oración
dada."""
    # Tokenizar
    inputs = tokenizer(sentence, return_tensors="pt")
    tokens =
tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])

    # Forward pass
    with torch.no_grad():
        outputs = model(**inputs)

    # Extraer atención: lista de tensores, uno por capa
    # Cada tensor tiene forma (batch, num_heads, seq_len, seq_len)
    attentions = outputs.attentions

    print(f"Oración: {sentence}")
    print(f"Tokens: {tokens}")
    print(f"Número de capas: {len(attentions)}")
    print(f"Forma de atención por capa: {attentions[0].shape}")

    # Visualización interactiva (funciona en Jupyter Notebook)
    head_view(attentions, tokens)

    return attentions, tokens

# Oración 1: Correferencia
sentence_1 = "El gato se sentó en la alfombra porque estaba
cansado"

```



```
attentions_1, tokens_1 = visualize_attention(sentence_1, model,
tokenizer)
```

Nota: Si no estás en Jupyter Notebook, puedes usar el siguiente código alternativo para inspeccionar numéricamente los pesos de atención:

```
def print_attention_weights(attentions, tokens, layer, head):
    """Imprime los pesos de atención de una cabeza específica."""
    att = attentions[layer][0, head].numpy()
    print(f"\n=== Capa {layer}, Cabeza {head} ===")
    print(f"{' ':>15}", end="")
    for t in tokens:
        print(f"{t:>12}", end="")
    print()
    for i, token in enumerate(tokens):
        print(f"{token:>15}", end="")
        for j in range(len(tokens)):
            print(f"{att[i][j]:>12.4f}", end="")
        print()

# Inspeccionar varias cabezas
for layer in [0, 5, 11]:
    for head in [0, 3, 7]:
        print_attention_weights(attentions_1, tokens_1, layer,
head)
```

Parte C: Análisis de Patrones (15 min)

Visualiza la atención para las siguientes oraciones y responde las preguntas:

Oración 1 - Correferencia:

```
sentence = "El gato se sentó en la alfombra porque estaba cansado"
```

- Busca una cabeza de atención donde "estaba" o "cansado" atienda fuertemente a "gato".
- En qué capa y cabeza la encuentras?
- Capa: ____ Cabeza: ____

Oración 2 - Estructura sintáctica:

```
sentence = "Los estudiantes que aprobaron el examen celebraron con sus amigos"
```

- Busca una cabeza donde "celebraron" atienda a "estudiantes" (sujeto del verbo, no a "examen").
- En qué capa y cabeza la encuentras?

- Capa: ____ Cabeza: ____

Oración 3 - Relaciones a larga distancia:

```
sentence = "La empresa que fundaron en Madrid hace diez años  
finalmente cerró"
```

- Busca una cabeza donde "cerro" atienda a "empresa" (saltando la cláusula relativa).
- Es más difícil de encontrar que en las oraciones anteriores? Por qué?

Oración 4 - Comparación de idiomas:

```
sentence_es = "El banco está cerca del río"  
sentence_en = "The bank is near the river"
```

- Compara los patrones de atención en ambos idiomas para el token "banco"/"bank" (palabra ambigua).
- Observas diferencias en qué tokens reciben atención? Documenta tus hallazgos.

Preguntas de Reflexión

1. En general, qué tipo de patrones observas en las capas tempranas (0-3) versus las capas profundas (9-11)?
 2. Algunas cabezas muestran un patrón de "atender al token anterior" o "atender al token [CLS]". Por qué serían útiles estos patrones aparentemente simples?
 3. Dado lo que observas, crees que cada cabeza se "especializa" en un tipo de relación lingüística, o es más sutil? Justifica tu respuesta.
-

Ejercicio 4: Diseño de un Transformer para un Caso de Uso

Metadata

- **Duración estimada:** 25 minutos
- **Tipo:** Diseño/Cálculo
- **Modalidad:** Grupal (3-4 personas)
- **Dificultad:** Avanzada
- **Prerrequisitos:** Comprensión de la arquitectura Transformer, conocimientos básicos de GPU y memoria

Contexto

En la práctica profesional, no siempre usamos modelos pre-entrenados. A veces necesitamos diseñar una arquitectura desde cero, adaptada a restricciones específicas de hardware, datos y tarea. Este ejercicio simula ese proceso de toma de decisiones.

Objetivo de Aprendizaje

- Seleccionar hiperparámetros de un Transformer de forma justificada
- Calcular el número total de parámetros de un modelo
- Comprender los trade-offs entre capacidad del modelo y restricciones de hardware
- Comparar decisiones de diseño con modelos conocidos

Enunciado

Escenario

Su equipo ha sido contratado para diseñar un modelo de lenguaje especializado en **documentación técnica en español** para una empresa de software. El modelo debe:

- Generar documentación de APIs a partir de código fuente
- Resumir changelogs largos
- Responder preguntas sobre la documentación existente

Restricciones

Recurso	Límite
GPU disponible	1x NVIDIA A100 (80 GB VRAM)
Datos de entrenamiento	~5 GB de texto (documentación técnica en español)
Tiempo de entrenamiento máximo	1 semana
Latencia de inferencia	< 100ms por token
Longitud máxima de contexto requerida	Documentos de hasta 4,000 tokens

Parte A: Selección de Hiperparámetros (10 min)

Completen la siguiente tabla justificando cada decisión:

Hiperparámetro	Valor Elegido	Justificación
Tipo de arquitectura (enc/dec/enc-dec)	—	—
d_model (dimensión del modelo)	—	—
num_heads (cabezas de atención)	—	—
num_layers (capas)	—	—
d_ff (dimensión feed-forward)	—	—
seq_length (longitud máxima)	—	—
vocab_size (tamaño del vocabulario)	—	—
dropout	—	—

Restricciones técnicas a considerar:

- d_model debe ser divisible por num_heads
- d_ff típicamente es $4 * d_model$
- El modelo debe caber en 80 GB de VRAM durante entrenamiento (modelo + gradientes + optimizador $\sim 4x$ tamaño del modelo en FP32)
- Vocabulario más grande = mejor representación de subpalabras, pero más parámetros en la capa de embedding

Parte B: Cálculo de Parámetros (10 min)

Calculen el número total de parámetros de su arquitectura usando las siguientes fórmulas:

1. Capa de Embedding:

```
Params_embedding = vocab_size * d_model
```

2. Por cada capa del Transformer:

Atención multi-cabeza:

$$\text{Params_attention} = 4 * d_model * d_model + 4 * d_model * (W_Q + W_K + W_V + W_O) + (bias_Q + bias_K + bias_V + bias_O)$$

Feed-Forward Network:

$$\text{Params_ffn} = d_model * d_ff + d_ff + d_ff * d_model + d_model * (W_1 + b_1 + W_2 + b_2)$$

Layer Normalization (x2 por capa):

$$\text{Params_layernorm} = 2 * (2 * d_model * (\gamma + \beta \text{ para cada LayerNorm}))$$

3. Capa de salida (cabeza de lenguaje):

$$\text{Params_output} = d_model * \text{vocab_size}$$

(nota: frecuentemente se comparten pesos con embedding)

Cálculo completo:

$$\begin{aligned} \text{Total} = & \text{Params_embedding} \\ & + \text{num_layers} * (\text{Params_attention} + \text{Params_ffn} + \\ & \text{Params_layernorm}) \\ & + \text{Params_output} \end{aligned}$$

Total = _____ parámetros

Convierte a millones (M) de parámetros: _____ M

Verificación de memoria:

Memoria modelo (FP32) = Total_params * 4 bytes = _____ GB
 Memoria entrenamiento (aprox.) = Memoria modelo * 4 = _____ GB
 Cabe en 80 GB? _____

Parte C: Comparación con Modelos Conocidos (5 min)

Comparen su diseño con estos modelos de referencia:

Modelo	Parámetros	d_model	Capas	Cabezas	d_ff	Vocab
BERT-base	110M	768	12	12	3072	30,522

GPT-2 Small	124M	768	12	12	3072	50,257
GPT-2 Medium	355M	1024	24	16	4096	50,257
LLaMA-7B	7B	4096	32	32	11,008	32,000

Preguntas:

1. Su modelo es más parecido en tamaño a cuál de los modelos de referencia?
2. Dado el tamaño del dataset (5 GB), creen que su modelo es demasiado grande, adecuado o demasiado pequeño? Justifiquen. (Regla general: se necesitan ~10-20 tokens por parámetro para un entrenamiento adecuado).
3. Si tuvieran el doble de VRAM, qué hiperparámetro cambiarían primero: más capas, mayor d_{model} , o mayor d_{ff} ? Por qué?
4. Considerarían usar weight tying (compartir pesos entre embedding y capa de salida)? Qué ventajas tendría en su caso?

Entregable

- Tabla de hiperparámetros con justificaciones
- Cálculo detallado de parámetros
- Comparación argumentada con modelos de referencia

Ejercicio Extra: Implementación de Self-Attention en Python

Metadata

- **Duración estimada:** 45 minutos (tarea para casa)
- **Tipo:** Programación
- **Modalidad:** Individual
- **Dificultad:** Avanzada
- **Prerrequisitos:** Python, NumPy, nociones básicas de PyTorch

Contexto

Implementar el mecanismo de atención desde cero consolida la comprensión teórica y prepara para trabajar con frameworks de deep learning. Este ejercicio es un puente entre la teoría matemática y la implementación práctica.

Objetivo de Aprendizaje

- Implementar single-head self-attention con NumPy puro
- Extender a multi-head attention
- Validar la implementación contra PyTorch
- Comprender las diferencias entre implementaciones educativas y de producción

Enunciado

Parte A: Single-Head Self-Attention con NumPy (20 min)

Implementa la función de atención escalada de producto punto:

```
import numpy as np

def softmax(x, axis=-1):
    """Softmax numéricamente estable."""
    # TODO: Implementar softmax estable
    # Pista: restar el máximo por fila para estabilidad numérica
    # exp_x = np.exp(x - np.max(x, axis=axis, keepdims=True))
    # return exp_x / np.sum(exp_x, axis=axis, keepdims=True)
    pass

def scaled_dot_product_attention(Q, K, V, mask=None):
    """
    Calcula la atención escalada de producto punto.

    Args:
        Q: Queries, forma (seq_len_q, d_k)
        K: Keys, forma (seq_len_k, d_k)
        V: Values, forma (seq_len_k, d_v)
        mask: Máscara opcional, forma (seq_len_q, seq_len_k)

    Returns:
        output: Resultado de la atención, forma (seq_len_q, d_v)
        attention_weights: Pesos de atención, forma (seq_len_q,
seq_len_k)
    """
    d_k = Q.shape[-1]

    # Paso 1: Calcular scores
    # TODO: scores = Q @ K^T

    # Paso 2: Escalar
    # TODO: scores = scores / sqrt(d_k)
```

```

# Paso 3: Aplicar máscara (si existe)
# TODO: si mask no es None, poner -infinito donde mask == 0
# Pista: scores = np.where(mask == 0, -1e9, scores)

# Paso 4: Softmax
# TODO: attention_weights = softmax(scores)

# Paso 5: Multiplicar por V
# TODO: output = attention_weights @ V

# return output, attention_weights
pass

# Test con las matrices del Ejercicio 1
Q = np.array([[1, 0], [0, 1], [1, 1]], dtype=np.float64)
K = np.array([[1, 0], [0, 1], [0.5, 0.5]], dtype=np.float64)
V = np.array([[1, 2], [3, 4], [5, 6]], dtype=np.float64)

output, weights = scaled_dot_product_attention(Q, K, V)
print("Pesos de atención:")
print(weights)
print("\nOutput:")
print(output)
print("\nVerificación: cada fila de pesos suma 1?",
np.allclose(weights.sum(axis=-1), 1.0))

```

Parte B: Multi-Head Attention con NumPy (15 min)

Extiende la implementación a multi-head attention:

```

class MultiHeadAttention:
    """Implementación de Multi-Head Attention con NumPy."""

    def __init__(self, d_model, num_heads, seed=42):
        """
        Args:
            d_model: Dimensión del modelo
            num_heads: Número de cabezas de atención
        """
        assert d_model % num_heads == 0, "d_model debe ser divisible por num_heads"

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

```



```

        # Inicializar pesos aleatoriamente (en la práctica se
aprenden)
        rng = np.random.RandomState(seed)
        scale = np.sqrt(2.0 / d_model)

        # TODO: Inicializar matrices de proyección
        # self.W_Q = rng.randn(d_model, d_model) * scale #
(d_model, d_model)
        # self.W_K = rng.randn(d_model, d_model) * scale
        # self.W_V = rng.randn(d_model, d_model) * scale
        # self.W_O = rng.randn(d_model, d_model) * scale
        pass

    def split_heads(self, x):
        """
        Divide la última dimensión en (num_heads, d_k).

        Args:
            x: forma (seq_len, d_model)

        Returns:
            forma (num_heads, seq_len, d_k)
        """
        seq_len = x.shape[0]
        # TODO: Reshape y transponer
        # x = x.reshape(seq_len, self.num_heads, self.d_k)
        # x = x.transpose(1, 0, 2) # (num_heads, seq_len, d_k)
        # return x
        pass

    def combine_heads(self, x):
        """
        Operación inversa de split_heads.

        Args:
            x: forma (num_heads, seq_len, d_k)

        Returns:
            forma (seq_len, d_model)
        """
        # TODO: Transponer y reshape
        # x = x.transpose(1, 0, 2) # (seq_len, num_heads, d_k)
        # seq_len = x.shape[0]
        # x = x.reshape(seq_len, self.d_model)
        # return x
        pass

    def forward(self, X, mask=None):

```

```

"""
    Forward pass de multi-head attention.

    Args:
        X: Entrada, forma (seq_len, d_model)
        mask: Máscara opcional

    Returns:
        output: forma (seq_len, d_model)
        attention_weights: forma (num_heads, seq_len, seq_len)
"""
# TODO: Implementar
# 1. Proyectar:  $Q = X @ W_Q$ ,  $K = X @ W_K$ ,  $V = X @ W_V$ 
# 2. Dividir en cabezas
# 3. Aplicar atención a cada cabeza
# 4. Concatenar cabezas
# 5. Proyección de salida:  $output = combined @ W_O$ 
pass

# Test
d_model = 8
num_heads = 2
seq_len = 4

```

```

mha = MultiHeadAttention(d_model, num_heads)

```

```

# Entrada simulada (4 tokens, dimensión 8)
np.random.seed(0)
X = np.random.randn(seq_len, d_model)

```

```

output, attn_weights = mha.forward(X)
print(f"Input shape: {X.shape}")
print(f"Output shape: {output.shape}")
print(f"Attention weights shape: {attn_weights.shape}")
print(f"\nPesos de atención, Cabeza 0:")
print(attn_weights[0])
print(f"\nPesos de atención, Cabeza 1:")
print(attn_weights[1])

```

Parte C: Validación contra PyTorch (10 min)

Compara tu implementación con la de PyTorch:

```

import torch
import torch.nn as nn

# Crear MultiHeadAttention de PyTorch

```

```

d_model = 8
num_heads = 2

mha_torch = nn.MultiheadAttention(embed_dim=d_model,
num_heads=num_heads, batch_first=False)

# Crear input
X_torch = torch.randn(4, 1, d_model) # (seq_len, batch, d_model)

# Forward pass de PyTorch
output_torch, weights_torch = mha_torch(X_torch, X_torch, X_torch)

print(f"PyTorch output shape: {output_torch.shape}")
print(f"PyTorch weights shape: {weights_torch.shape}")

# Preguntas:
# 1. Las shapes de salida de tu implementación coinciden con las de
PyTorch?
# 2. Los valores numéricos no coincidirán exactamente. Por qué?
# (Pista: pesos iniciales diferentes)
# 3. Qué diferencias de API observas entre tu implementación y la de
PyTorch?

```

Preguntas Finales

1. En tu implementación, qué pasaría si no aplicaras el escalado por $\sqrt{d_k}$? Pruébalo con $d_k = 64$ y observa los pesos de atención.
2. Implementa una máscara causal (triangular inferior) y verifícala con una secuencia de ejemplo. Qué efecto tiene en los pesos de atención?
3. Tu implementación usa NumPy (CPU). Qué cambios serían necesarios para hacerla eficiente en GPU? Menciona al menos 3 consideraciones.

Entregable

- Código completo funcionando (archivo `.py` o notebook `.ipynb`)
- Output de los tests mostrando formas y valores
- Respuestas a las preguntas finales

Soluciones de Referencia

Ver solución Ejercicio 1 - Cálculo Manual de Self-Attention

Paso 1: Scores = Q * K^T

```
K^T = [[1, 0, 0.5],  
       [0, 1, 0.5]]
```

```
Scores = Q @ K^T
```

```
Fila 0: [1,0] @ [[1,0,0.5],[0,1,0.5]] = [1*1+0*0, 1*0+0*1,  
1*0.5+0*0.5] = [1, 0, 0.5]
```

```
Fila 1: [0,1] @ [[1,0,0.5],[0,1,0.5]] = [0*1+1*0, 0*0+1*1,  
0*0.5+1*0.5] = [0, 1, 0.5]
```

```
Fila 2: [1,1] @ [[1,0,0.5],[0,1,0.5]] = [1*1+1*0, 1*0+1*1,  
1*0.5+1*0.5] = [1, 1, 1]
```

```
Scores = [[1, 0, 0.5],  
          [0, 1, 0.5],  
          [1, 1, 1 ]]
```

El par con mayor compatibilidad: Token 0 consigo mismo (1.0), Token 1 consigo mismo (1.0), y Token 2 con todos (1.0 cada uno).

Paso 2: Scaled Scores = Scores / sqrt(2)

```
sqrt(2) = 1.4142
```

```
Scaled_Scores = [[0.7071, 0.0000, 0.3536],  
                 [0.0000, 0.7071, 0.3536],  
                 [0.7071, 0.7071, 0.7071]]
```

El escalado previene que los valores sean demasiado grandes, lo que haría que softmax produzca distribuciones casi one-hot, dificultando el flujo de gradientes (gradientes muy pequeños en las regiones saturadas de softmax).

Paso 3: Softmax por filas

Fila 0:

```
exp(0.7071) = 2.0281  
exp(0.0000) = 1.0000  
exp(0.3536) = 1.4243  
Suma = 4.4524  
Softmax = [0.4555, 0.2246, 0.3199]
```

Fila 1:

```
exp(0.0000) = 1.0000  
exp(0.7071) = 2.0281
```

```
exp(0.3536) = 1.4243
Suma = 4.4524
Softmax = [0.2246, 0.4555, 0.3199]
```

Fila 2:

```
exp(0.7071) = 2.0281
exp(0.7071) = 2.0281
exp(0.7071) = 2.0281
Suma = 6.0844
Softmax = [0.3333, 0.3333, 0.3333]
```

```
Attention_Weights = [[0.4555, 0.2246, 0.3199],
                     [0.2246, 0.4555, 0.3199],
                     [0.3333, 0.3333, 0.3333]]
```

Cada fila suma 1.0 porque softmax produce una distribución de probabilidad. Esto asegura que la salida sea un promedio ponderado (convex combination) de los valores V.

Paso 4: Output = Attention_Weights @ V

```
V = [[1, 2],
     [3, 4],
     [5, 6]]
```

```
Fila 0: [0.4555*1 + 0.2246*3 + 0.3199*5, 0.4555*2 + 0.2246*4 +
0.3199*6]
        = [0.4555 + 0.6738 + 1.5995, 0.9110 + 0.8984 + 1.9194]
        = [2.7288, 3.7288]
```

```
Fila 1: [0.2246*1 + 0.4555*3 + 0.3199*5, 0.2246*2 + 0.4555*4 +
0.3199*6]
        = [0.2246 + 1.3665 + 1.5995, 0.4492 + 1.8220 + 1.9194]
        = [3.1906, 4.1906]
```

```
Fila 2: [0.3333*1 + 0.3333*3 + 0.3333*5, 0.3333*2 + 0.3333*4 +
0.3333*6]
        = [0.3333 + 0.9999 + 1.6665, 0.6666 + 1.3332 + 1.9998]
        = [2.9997, 3.9996]
        ~ [3.0, 4.0]
```

```
Output = [[2.7288, 3.7288],
          [3.1906, 4.1906],
          [3.0000, 4.0000]]
```

Respuestas a las preguntas de reflexión

- 1. El Token 2 tiene pesos de atención iguales (1/3 cada uno) porque su query [1,1] tiene la misma compatibilidad con todos los keys. Su salida [3.0, 4.0] es exactamente el promedio de los tres vectores V: $(V[0]+V[1]+V[2])/3 = ([1,2]+[3,4]+[5,6])/3 = [3,4]$.
- 2. Con $Q[0] = [10, 0]$, el score sin escalar para el Key 0 sería 10, mucho mayor que para los demás. Aún con escalado, softmax produciría una distribución casi one-hot concentrada en el Token 0. Esto muestra cómo vectores query grandes "sharpean" la atención.
- 3. Proyecciones separadas permiten al modelo aprender diferentes representaciones para el rol de "pregunta" (Q), "respuesta disponible" (K) y "contenido a recuperar" (V). El mismo token puede ser relevante como key pero aportar información diferente como value.

Ver solución Ejercicio 2 - Análisis de Arquitecturas

Parte A: Tabla Completada

Modelo	Organización	Tipo de Arquitectura	Caso de Uso Principal
BERT	Google	Encoder-only	Comprensión/clasificación de texto
GPT-2	OpenAI	Decoder-only	Generación de texto
GPT-4	OpenAI	Decoder-only	Generación de texto conversacional y de propósito general
T5	Google	Encoder-decoder	Traducción / tareas seq2seq
Claude 3.5	Anthropic	Decoder-only	Generación de texto conversacional y de propósito general
LLaMA 3	Meta	Decoder-only	Generación de texto
BART	Meta (Facebook AI)	Encoder-decoder	Resumen y generación condicional
RoBERTa	Meta (Facebook AI)	Encoder-only	Comprensión/clasificación de texto
Mistral 7B	Mistral AI	Decoder-only	Generación de texto

Gemini	Google DeepMind	Decoder-only	Generación de texto conversacional y de propósito general
ALBERT	Google	Encoder-only	Comprensión eficiente de texto
Whisper	OpenAI	Encoder-decoder	Reconocimiento automático de habla (ASR)

Parte B: Análisis de Tendencias

- Conteo:**
 - Encoder-only: 3 (BERT, RoBERTa, ALBERT)
 - Decoder-only: 6 (GPT-2, GPT-4, Claude 3.5, LLaMA 3, Mistral 7B, Gemini)
 - Encoder-decoder: 3 (T5, BART, Whisper)
- Convergencia hacia decoder-only:** Se debe principalmente a que decoder-only escala mejor con datos y cómputo. Además, mediante instrucciones y fine-tuning, los decoder-only pueden realizar tareas de comprensión y seq2seq que antes requerían encoder-only o encoder-decoder. La simplicidad arquitectónica facilita el escalado.
- Escala vs. arquitectura:** Ambas importan. GPT-2 (decoder-only, 1.5B) no superaba a BERT (encoder-only, 110M) en tareas de clasificación a pesar de tener más parámetros. Sin embargo, a escala suficiente (GPT-3, GPT-4), los decoder-only superan a los encoder-only incluso en tareas de comprensión. La arquitectura define qué es posible; la escala define qué tan bien se ejecuta.

Parte C: Respuestas de Profundidad

- Enmascaramiento causal:** En generación, el modelo produce tokens uno a uno; no puede "ver el futuro" porque aún no existe. La máscara causal imita esta condición durante entrenamiento. BERT, en cambio, se entrena con masked language modeling (MLM), donde predice tokens enmascarados usando contexto bidireccional, lo que requiere ver tokens tanto a izquierda como a derecha.
- Encoder-decoder vs decoder-only para traducción:** Un decoder-only realiza traducción tratándola como continuación de texto: dado el prompt "Traduce al inglés: [texto en español]", genera la traducción autorregresivamente. El compromiso es que pierde la ventaja del encoding bidireccional de la entrada y usa tokens de contexto para la instrucción, reduciendo espacio útil.
- Whisper:** La señal de audio (espectrograma) y el texto son modalidades fundamentalmente diferentes. El encoder procesa la representación acústica de forma bidireccional (puede "ver" todo el audio), y el decoder genera texto condicionado en esa representación. Un decoder-only tendría que serializar audio y texto en una sola secuencia, lo que es menos eficiente y pierde la ventaja del procesamiento bidireccional del audio.

Ver solución Ejercicio 4 - Diseño de un Transformer

Diseño Propuesto

Hiperparámetro	Valor Elegido	Justificación
Tipo de arquitectura	Decoder-only	Más simple de escalar, puede hacer generación y comprensión. Las tres tareas (generar docs, resumir, QA) son naturales para decoder-only con prompting adecuado
d_model	1024	Balance entre capacidad y eficiencia. Suficiente para capturar semántica de documentación técnica
num_heads	16	16 cabezas con d_k = 64 cada una (1024/16). Permite capturar múltiples tipos de relaciones simultáneamente
num_layers	24	Profundidad moderada que permite buen aprendizaje de representaciones jerárquicas
d_ff	4096	Convención estándar de 4 * d_model
seq_length	4096	Cumple el requisito de 4,000 tokens con margen
vocab_size	32,000	Similar a LLaMA, buen balance para español. Se usaría SentencePiece/BPE entrenado en el corpus
dropout	0.1	Valor estándar; dataset moderado requiere algo de regularización

Cálculo de Parámetros

Embedding:

$32,000 * 1,024 = 32,768,000$

Por capa de Transformer:

Atención: $4 * 1024 * 1024 + 4 * 1024 = 4,198,400$
FFN: $1024 * 4096 + 4096 + 4096 * 1024 + 1024 = 8,393,728$
LayerNorm: $2 * (2 * 1024) = 4,096$
Total por capa: $12,596,224$

24 capas:


```
24 * 12,596,224 = 302,309,376
```

Capa de salida (con weight tying, compartimos con embedding):

```
0 (se reutilizan los pesos de embedding)
```

Total:

```
32,768,000 + 302,309,376 = 335,077,376 parámetros ~ 335M
```

Verificación de memoria:

```
Memoria FP32: 335M * 4 bytes = 1.34 GB
```

```
Memoria entrenamiento: 1.34 * 4 = 5.36 GB
```

```
Cabe en 80 GB? Sí, con gran margen.
```

Comparación

El modelo (~335M) es comparable en tamaño a GPT-2 Medium (355M). Dado el dataset de 5 GB (~1.3B tokens aproximadamente), la relación tokens/parámetro sería ~3.9, lo cual es bajo según la regla general de 10-20 tokens por parámetro. Opciones: (a) aumentar datos con data augmentation o datos sintéticos, (b) reducir el modelo a ~100M parámetros, o (c) aceptar que el modelo podría no converger completamente y usar técnicas de regularización fuertes.

Con el doble de VRAM, convendría aumentar `d_model` antes que las capas, ya que la dimensión del modelo tiende a mejorar la representación de forma más eficiente que la profundidad, especialmente en datasets moderados.

Ver solución Ejercicio Extra - Implementación de Self-Attention

Parte A: Single-Head Attention

```
import numpy as np
```

```
def softmax(x, axis=-1):  
    """Softmax numéricamente estable."""  
    exp_x = np.exp(x - np.max(x, axis=axis, keepdims=True))  
    return exp_x / np.sum(exp_x, axis=axis, keepdims=True)
```

```
def scaled_dot_product_attention(Q, K, V, mask=None):  
    """Calcula la atención escalada de producto punto."""  
    d_k = Q.shape[-1]
```

```
    # Paso 1: Scores  
    scores = Q @ K.T
```

```

# Paso 2: Escalar
scores = scores / np.sqrt(d_k)

# Paso 3: Máscara
if mask is not None:
    scores = np.where(mask == 0, -1e9, scores)

# Paso 4: Softmax
attention_weights = softmax(scores, axis=-1)

# Paso 5: Multiplicar por V
output = attention_weights @ V

return output, attention_weights

# Test
Q = np.array([[1, 0], [0, 1], [1, 1]], dtype=np.float64)
K = np.array([[1, 0], [0, 1], [0.5, 0.5]], dtype=np.float64)
V = np.array([[1, 2], [3, 4], [5, 6]], dtype=np.float64)

output, weights = scaled_dot_product_attention(Q, K, V)
print("Pesos de atención:")
print(np.round(weights, 4))
print("\nOutput:")
print(np.round(output, 4))

```

Salida esperada:

```

Pesos de atención:
[[0.4555 0.2246 0.3199]
 [0.2246 0.4555 0.3199]
 [0.3333 0.3333 0.3333]]

```

```

Output:
[[2.7288 3.7288]
 [3.1906 4.1906]
 [3.      4.      ]]

```

Parte B: Multi-Head Attention

```

class MultiHeadAttention:
    def __init__(self, d_model, num_heads, seed=42):
        assert d_model % num_heads == 0
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

```

```

        rng = np.random.RandomState(seed)
        scale = np.sqrt(2.0 / d_model)
        self.W_Q = rng.randn(d_model, d_model) * scale
        self.W_K = rng.randn(d_model, d_model) * scale
        self.W_V = rng.randn(d_model, d_model) * scale
        self.W_O = rng.randn(d_model, d_model) * scale

    def split_heads(self, x):
        seq_len = x.shape[0]
        x = x.reshape(seq_len, self.num_heads, self.d_k)
        x = x.transpose(1, 0, 2)  # (num_heads, seq_len, d_k)
        return x

    def combine_heads(self, x):
        x = x.transpose(1, 0, 2)  # (seq_len, num_heads, d_k)
        seq_len = x.shape[0]
        x = x.reshape(seq_len, self.d_model)
        return x

    def forward(self, X, mask=None):
        # Proyecciones lineales
        Q = X @ self.W_Q
        K = X @ self.W_K
        V = X @ self.W_V

        # Dividir en cabezas
        Q_heads = self.split_heads(Q)
        K_heads = self.split_heads(K)
        V_heads = self.split_heads(V)

        # Atención por cabeza
        all_outputs = []
        all_weights = []
        for i in range(self.num_heads):
            out, w = scaled_dot_product_attention(
                Q_heads[i], K_heads[i], V_heads[i], mask
            )
            all_outputs.append(out)
            all_weights.append(w)

        # Concatenar cabezas
        multi_output = np.stack(all_outputs, axis=0)  # (num_heads,
seq_len, d_k)
        attention_weights = np.stack(all_weights, axis=0)

        combined = self.combine_heads(multi_output)

```

```

        # Proyección de salida
        output = combined @ self.W_O

    return output, attention_weights

```

Máscara Causal

```

def create_causal_mask(seq_len):
    """Crea una máscara triangular inferior."""
    mask = np.tril(np.ones((seq_len, seq_len)))
    return mask

# Ejemplo
mask = create_causal_mask(4)
print("Máscara causal:")
print(mask)
# [[1. 0. 0. 0.]
#  [1. 1. 0. 0.]
#  [1. 1. 1. 0.]
#  [1. 1. 1. 1.]]

# Usar con atención
Q_test = np.random.randn(4, 2)
K_test = np.random.randn(4, 2)
V_test = np.random.randn(4, 2)

output_masked, weights_masked =
scaled_dot_product_attention(Q_test, K_test, V_test, mask)
print("\nPesos con máscara causal:")
print(np.round(weights_masked, 4))
# Observar: la triangular superior es 0 (cada token solo atiende a
tokens previos e incluido el mismo)

```